

API REST para la gestión de registro de estudiantes y profesores de una Universidad.

Una universidad necesita una API que ayude al registro de estudiantes, carreras, profesores y materias de forma básica:

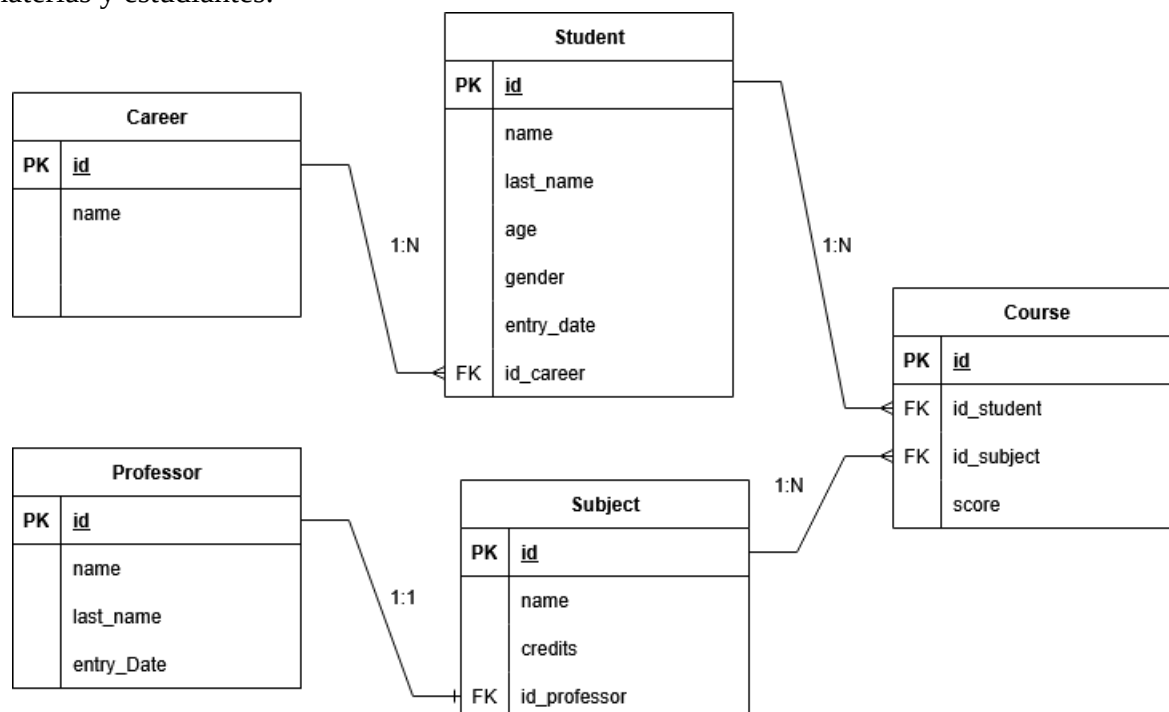
- Un estudiante puede registrarse en diferentes materias.
- Una materia puede tener varios estudiantes.
- Un estudiante pertenece a una carrera específica
- Un profesor solo puede impartir una materia.

En este proyecto se planea evidenciar:

- El uso de Spring Boot con JPA para manejo de base de datos.
- El uso de base de datos relacional MySQL.
- Exponer endpoints RESTful para realizar CRUDs (GET, POST, PUT, DELETE o los métodos que se consideren necesarios).
- Uso de DTOs para separar modelo de dominio y representación externa.
- Manejo adecuado de errores con ResponseEntity, código HTTP correcto (status code) mensajes claros.
- Uso de lambdas o streams
- Uso de Enum y Exceptions
- Organización modular (Service, Repository, controller).

Solución:

Para cumplir con lo solicitado, se define un total de 5 entidades que serán Career, Student, Professor, Subject y Course, las cuales representan a las carreras, estudiantes, profesores, materias y cursos, respectivamente, en donde Course es la entidad que ayuda a la relación de muchos a muchos de materias y estudiantes.



Manejo de datos

En la imagen anterior se especificaron las entidades, sus atributos y las relaciones que tienen unas con otras. La integridad de datos está definida de la siguiente manera:

- A la hora de registrar un elemento en la base de datos, el id de un registro es dado por el sistema. Las llaves foráneas son obligatorias, además de que una combinación de llaves foráneas solo puede dar una sola vez; ejemplo: en la entidad Course, un estudiante no puede estar registrado dos veces en la misma materia.
- La eliminación de datos solo será efectiva si la PK de un registro no está enlazada a la FK de otro registro; esto es con el fin de mantener la integridad de los datos, así que la función CASCADE no aplicará para esta API.
- La actualización de los registros está permitida para todos los campos, menos para los campos PK y para los FK de la entidad Course.

Endpoints RESTful

Para este proyecto, el servidor será localhost y su puerto 8080; por lo tanto, la URL se verá de la siguiente forma:

<http://localhost:8080/api/{endPoint}>

Donde el endPoint puede ser /careers, /students, /professors, /subjects o /courses.

Servicio GET & POST

<http://localhost:8080/api/{endPoint}>: El servicio GET traerá todos los registros de una entidad de la base de datos.

<http://localhost:8080/api/{endPoint}>: El servicio POST guardará solo un registro de una entidad en la base de datos.

Servicio PUT & DELETE

<http://localhost:8080/api/{endPoint}/{id}>: Para el servicio PUT se necesita como parámetro el ID del registro; en caso de existir, se actualizará el dato; de lo contrario, se lanzará un response entity 400.

<http://localhost:8080/api/{endPoint}/{id}>: Para el servicio DELETE se necesita como parámetro el ID del registro; en caso de existir, se actualizará el dato; de lo contrario, se lanzará un response entity 400.